

Authentication and Authorization Mechanisms

[Jan Pluskal <pluskal@vut.cz>]

Lecture Agenda

1. Authentication vs. Authorization — core concepts and terminology
2. Authentication Methods — passwords, MFA, passkeys, SSO
3. Authorization Models — DAC, MAC, RBAC, ABAC
4. Session Management and Token-Based Authentication (JWT)
5. Practical On-Premise Implementations — LDAP, Kerberos, SAML, OAuth 2.0, OIDC
6. Common Issues and Security Best Practices
7. Real-world Case Studies
8. Open Source Self-Hosted Identity Providers (Authentik)

What is Authentication?

Definition:

- **Authentication** is the process of verifying the identity of a user, application, or device.
- It answers the question:
 "Are you really who you claim to be?"

Examples:

- Username/password login
- Biometrics (fingerprint, facial recognition)
- Multi-factor Authentication (MFA)

What is Authorization?

Definition:

- **Authorization** determines the permissions and privileges granted to an authenticated user, application, or device.
- It answers the question:
 "Are you allowed to access or perform this action?"

Examples:

- Accessing files on a server
- Modifying user permissions
- Running administrative commands

Authentication vs. Authorization

Authentication	Authorization
Verifies user <i>identity</i>	Defines user <i>permissions</i>
Occurs <i>before</i> authorization	Occurs <i>after</i> authentication
Usually <i>requires user input</i> (e.g., credentials)	Managed through <i>rules or policies</i>
Example: Entering a username and password	Example: Checking user role before accessing a file

Why are Authentication and Authorization Important?

- Protect sensitive information from unauthorized access.
- Essential for maintaining system security and integrity.
- Compliance with regulations and standards (e.g., GDPR, HIPAA).
- Foundation of robust security architecture.

Common Misconceptions

- **Authentication $\neq$ Authorization:**
They serve distinct roles, even though often linked.
- **Complexity $\neq$ Security:**
Complex systems can introduce vulnerabilities if poorly implemented.
- **Single security measure is sufficient:**
Effective security relies on layers (defense-in-depth).

Authentication Methods and Techniques

Overview of Authentication Methods

- Password-Based Authentication
- Multi-Factor Authentication (MFA)
- Passkeys and Passwordless Authentication
- Single Sign-On (SSO)

Password-Based Authentication

Most Common Method:

- Users provide username/password credentials.
- Simple but vulnerable if mismanaged.

Key Concepts:

- **Hashing:** Storing passwords securely (bcrypt, Argon2, PBKDF2).
- **Salting:** Adding randomness to prevent rainbow-table attacks.

Password Best Practices

- **Length over Complexity:**

Prioritize length (minimum 8 characters, support up to 64+). Per NIST SP 800-63B, mandatory character-class rules (forced uppercase, symbols) are **discouraged** — they frustrate users without meaningfully improving security. Check passwords against known-breached lists instead.

- **Regular Rotation:**

Change passwords immediately upon suspected compromise (mandatory periodic rotation is no longer recommended per NIST SP 800-63B).

- **Secure Storage:**

Never store plain-text passwords; use strong hash algorithms.

- **Password Managers:**

Encourage the use of secure password management tools.

Limitations of Password Authentication

- Susceptible to brute force and dictionary attacks.
- Vulnerable to phishing and social engineering.
- Password reuse can lead to cascading security breaches.

Key Mitigations:

- **Account lockout / rate limiting:** Temporarily lock or throttle responses after repeated failed attempts.
- **Breached-password checking:** Reject passwords found in known breach lists (e.g., via the HaveIBeenPwned API).

Multi-Factor Authentication (MFA)

Definition:

Using two or more independent authentication factors.

Authentication Factors:

- **Something you know:** password or PIN.
- **Something you have:** hardware tokens, smartphone apps.
- **Something you are:** biometric traits (fingerprint, facial recognition).

Types of MFA and Practical Examples

Type	Example Tools
Authenticator Apps	Google Authenticator, Authy
Hardware Tokens	YubiKey, RSA ID Plus
Biometric Methods	Fingerprint scanners, Face ID

Strengths and Weaknesses of MFA

Strengths:

- Enhances security significantly.
- Reduces risks associated with compromised passwords.

Weaknesses:

- May introduce usability challenges.
- Dependency on devices (tokens, smartphones).
- Biometric data privacy concerns.

Passkeys and Passwordless Authentication

The Problem with Passwords:

- Stolen, phished, reused, or guessed.
- Even with OTP-based MFA, real-time phishing proxies can relay codes before expiry.

Passwordless Authentication:

- Authenticate without a traditional password.
- Methods: biometrics only, hardware security keys, magic email links, or passkeys.

Passkeys / FIDO2 / WebAuthn

What are Passkeys?

- Cryptographic key pairs stored in a device's hardware-backed secure enclave.
- **Private key** never leaves the device; **public key** is registered on the server.
- Phishing-resistant: the credential is cryptographically bound to the specific origin (domain).

Standards:

- **FIDO2** = **WebAuthn** (W3C browser API) + **CTAP** (Client-to-Authenticator Protocol).
- Supported natively: Apple (Touch/Face ID), Google (Android, Chrome), Microsoft (Windows Hello).

Passkey Authentication Flow

1. User visits site and selects "Sign in with Passkey."
2. Browser/OS presents stored passkeys for that specific domain.
3. User authenticates locally via biometric or PIN — **nothing sent to the server.**
4. Device signs a server-issued challenge with the private key.
5. Server verifies the signature with the stored public key.
6. Access granted — **no password ever transmitted.**

Key advantage: A phishing site operates on a different domain — the credential binding fails automatically.

Single Sign-On (SSO)

Definition:

Allows users to log in once and access multiple applications seamlessly.

Benefits:

- Simplifies user experience.
- Centralized credential management.
- Reduces password fatigue.

Common SSO Protocols

- **SAML:** XML-based framework used primarily in enterprise environments.
- **OpenID Connect (OIDC):** Identity layer built on top of OAuth 2.0; the primary open-standard SSO/authentication protocol.

⚠ **Common misconception:** *OAuth 2.0 is a delegated authorization framework, **not** an authentication or SSO protocol. It grants access to resources without sharing passwords. Authentication and identity are handled by **OIDC**, which extends OAuth 2.0 with an identity layer.*

Security Considerations for SSO

- Single point of failure: Compromise impacts multiple services.
- Proper configuration and maintenance critical for security.
- Secure handling of authentication tokens required.

Summary of Authentication Methods

Method	Security Level	Usability Level	Implementation Effort
Password-only	Low	High	Low
Password + MFA (OTP)	High	Medium	Medium
Passkeys (FIDO2)	Very High	High	Medium
SSO	Medium to High	High	Medium to High

Authorization Models

Introduction to Authorization

What is Authorization?

- Defines **permissions and access levels** for authenticated users.
- Controls who can access resources and perform actions.

Goal: Ensure appropriate access in accordance with policies and roles.

Common Authorization Models

- Discretionary Access Control (**DAC**)
- Mandatory Access Control (**MAC**)
- Role-Based Access Control (**RBAC**)
- Attribute-Based Access Control (**ABAC**)

Discretionary Access Control (DAC)

Definition:

- Resource owners define and control access permissions.

Characteristics:

- Flexible, intuitive control by resource owners.
- Commonly implemented via file permissions (UNIX, Windows).

Example:

- User-defined permissions in a file-sharing system.

Advantages and Limitations of DAC

Advantages:

- Easy for end-users to manage.
- Intuitive, straightforward implementation.

Limitations:

- Security risks if users mishandle permissions.
- Susceptible to privilege escalation.

Mandatory Access Control (MAC)

Definition:

- Permissions defined by a central authority (administrator).
- Users/resources have labels (classification levels).

Characteristics:

- Highly secure, rigid structure.
- Commonly used in military, government settings.

Example:

- Classified documents ("Top Secret," "Confidential").

Advantages and Limitations of MAC

Advantages:

- Strong security enforcement.
- Minimizes risk of user-based errors.

Limitations:

- Less flexible and scalable.
- Complexity in administration and maintenance.

Role-Based Access Control (RBAC)

Definition:

- Permissions assigned to roles, not individual users.
- Users inherit permissions by being assigned to roles.

Characteristics:

- Widely adopted in enterprise environments.
- Simplifies large-scale permission management.

Example:

- Employee roles ("Admin," "Developer," "HR").

Advantages and Limitations of RBAC

Advantages:

- Simplifies management of large user groups.
- Clear, structured approach.

Limitations:

- Inflexible when individual customization required.
- Role explosion if poorly managed.

Attribute-Based Access Control (ABAC)

Definition:

- Permissions granted based on attributes and context.
- Attributes may include user roles, resource type, time, location.

Characteristics:

- Highly dynamic and context-sensitive.
- Fine-grained access control.

Example:

- Allow access to sensitive files only during work hours, from corporate devices.

Advantages and Limitations of ABAC

Advantages:

- Very flexible, granular control.
- Adaptive to changing conditions.

Limitations:

- Complex implementation and management.
- Higher initial setup effort.

Comparing Authorization Models

Model	Flexibility	Complexity	Use-Case
DAC	High	Low	Small, user-managed systems
MAC	Low	High	Highly secure environments
RBAC	Medium	Medium	Enterprise, structured org.
ABAC	Very High	High	Complex, adaptive scenarios

Choosing an Authorization Model

Consider:

- **Security requirements:** Level of strictness needed.
- **Complexity:** Management resources available.
- **Scalability:** Future growth or changes.
- **Use-case:** Organization size, user types, resource sensitivity.

Session Management and Token-Based Authentication

What is Session Management?

- **Process of securely maintaining user state** after authentication.
- Allows users to interact without constant re-authentication.

Common session tracking methods:

- **Server-side sessions**
- **Client-side sessions (cookies)**
- **Token-based sessions**

Server-side Session Management

How it works:

- Server creates a session ID upon successful login.
- Session ID stored on server; client stores reference (usually cookie).
- Client sends session ID on each request.

Advantages:

- Greater control and security (session stored server-side).
- Easier invalidation.

Disadvantages:

- Potentially resource-intensive (server storage).
- Scalability challenges (requires shared session storage).

Client-side Sessions (Cookies)

How it works:

- Authentication info stored in client cookies.
- Cookies sent by client in each request.

Security Considerations:

- Must use **HttpOnly** and **Secure** flags.
- Vulnerable to theft if not protected (e.g., via XSS attacks).
- Limit cookie lifespan to reduce exposure.

Token-Based Authentication

Introduction:

- Stateless authentication using cryptographically-signed tokens.
- Tokens are typically structured as **JSON Web Tokens (JWT)**.

Use Cases:

- REST APIs, Single Page Applications (SPAs), and microservices.

JSON Web Tokens (JWT) Overview

JWT structure:

```
header.payload.signature
```

- **Header:** Token type and signing algorithm (e.g., RS256).
- **Payload:** Claims, such as user data, expiration (`exp`), issuer (`iss`).
- **Signature:** Ensures integrity and authenticity.

JWT Authentication Flow

1. User logs in; server issues signed JWT.
2. Client stores JWT (browser storage or cookies).
3. Client attaches JWT in requests (e.g., Authorization header).
4. Server validates JWT for each request.

Advantages and Limitations of JWT

Advantages:

- Stateless, scalable, and flexible.
- Simple to use across distributed services.

Limitations:

- Token revocation and invalidation complexity.
- Risk if tokens exposed or improperly managed.

JWT Security Best Practices

- ✓ Use strong signature algorithms (e.g., RS256).
- ✓ Short-lived tokens; refresh tokens securely.
- ✓ Minimal payload; avoid sensitive data in JWTs.
- ✓ Secure token transmission (HTTPS only).

Common Session Management Vulnerabilities

- **Session Hijacking:** Secure cookies and encryption.
- **Session Fixation:** Regenerate session IDs post-login.
- **Token Leakage:** Protect tokens from XSS, CSRF attacks.

Session Management Checklist

- ✓ Implement secure flags (`HttpOnly`, `Secure`) on cookies.
- ✓ Ensure secure transport (HTTPS).
- ✓ Rotate session IDs and tokens regularly.
- ✓ Monitor and log session activities for anomalies.

Summary

- Effective session management is essential for a secure user experience.
- Token-based systems (JWT) are highly scalable but require careful handling.
- Combine best practices to maximize both security and usability.

Practical Implementations in On-Premise Infrastructure

Overview

Practical implementations of:

- **LDAP** (Lightweight Directory Access Protocol)
- **SAML** (Security Assertion Markup Language)
- **OAuth 2.0**
- **OpenID Connect (OIDC)**

LDAP (Lightweight Directory Access Protocol)

What is LDAP?

- Protocol for accessing and managing directory services.
- Centralized user authentication, authorization, and information storage.

Typical usage:

- Authentication backend for internal systems.
- Central management of user identities.

LDAP Structure

- **Directory Tree:** hierarchical storage of objects.
- **Distinguished Names (DNs):** Unique identifiers for objects.
- **Attributes:** Data associated with each object (e.g., username, email, roles).

Example DN:

```
cn=John Doe,ou=Users,dc=example,dc=com
```

LDAP Authentication Workflow

1. User provides credentials (username/password).
2. Application performs LDAP bind operation.
3. LDAP server verifies credentials.
4. LDAP returns success/failure and user attributes.

LDAP Implementation Best Practices

- ✓ Use secure protocols (LDAPS or TLS).
- ✓ Enforce strong authentication methods (e.g., Kerberos).
- ✓ Limit LDAP query permissions strictly.
- ✓ Regular maintenance (backup, indexing, cleanup).

Kerberos — Authentication for Directory Services

What is Kerberos?

- Network authentication protocol using symmetric-key cryptography and a trusted third party (KDC).
- Default authentication protocol in Microsoft Active Directory.

Key Components:

- **KDC (Key Distribution Center):** Central trusted server with two roles:
 - **Authentication Server (AS):** Issues Ticket-Granting Tickets (TGTs) after initial login.
 - **Ticket-Granting Server (TGS):** Issues service tickets from a valid TGT.
- **Ticket:** Short-lived, encrypted token granting access to a specific service — credentials never travel the network again.

LDAP + Kerberos in practice:

- LDAP handles *directory queries* (who is the user, what groups do they belong to?).
- Kerberos handles *authentication* (proving identity via tickets, not passwords on the wire).

- Together they form the backbone of Active Directory and Linux SSO setups (e.g., SSSD, MIT Kerberos).

SAML (Security Assertion Markup Language)

What is SAML?

- XML-based protocol for Single Sign-On (SSO).
- Enables authentication between multiple internal applications securely.

Roles involved:

- **Identity Provider (IdP)**: Authenticates users.
- **Service Provider (SP)**: Hosts protected applications.

Practical SAML Workflow

1. User attempts to access Service Provider application.
2. Redirected to Identity Provider for authentication.
3. User authenticates (often via LDAP or internal credentials).
4. IdP issues signed **SAML Assertion** to SP.
5. SP validates assertion and grants access.

SAML Assertion Structure (Simplified)

```
<Assertion>  
  <Issuer>https://idp.example.com</Issuer>  
  <Subject>User123</Subject>  
  <Conditions> (Validity, Audience, etc.) </Conditions>  
  <AttributeStatement>  
    <Attribute Name="Role">Admin</Attribute>  
  </AttributeStatement>  
  <Signature> (XML Signature) </Signature>  
</Assertion>
```

SAML Implementation Best Practices

- ✓ Use HTTPS for assertion exchanges.
- ✓ Sign and encrypt assertions securely.
- ✓ Regularly update certificates and metadata.
- ✓ Limit assertion validity to minimize risk exposure.

OAuth 2.0 in On-Premise Infrastructure

When to use OAuth on-premise?

- Secure delegated access between internal services.
- Integrate third-party internal apps without sharing passwords.

OAuth components:

- **Authorization Server:** Issues tokens.
- **Resource Server:** Protects resources.
- **Client:** Application requesting access.
- **Resource Owner:** Typically end-user.

OAuth Authorization Code Flow (On-Premise)

1. User initiates login via application.
2. Redirected to internal authorization server.
3. User authenticates internally (LDAP or SAML-based).
4. Authorization server returns authorization code.
5. Application exchanges code for access token.
6. Application accesses resource server using token.

OAuth Implementation Best Practices

- ✓ Use Authorization Code Flow with PKCE (Proof Key for Code Exchange).
- ✓ Implement HTTPS strictly.
- ✓ Limit token lifetime and scope.
- ✓ Securely store and manage tokens (encrypted storage).

Additional OAuth 2.0 Grant Types

Grant Type	Use Case
Authorization Code + PKCE	Web and mobile apps acting on behalf of a user (recommended default)
Client Credentials	Machine-to-machine (M2M) service calls — no user involved
Device Code	Input-limited devices (smart TVs, IoT) — user authenticates on a separate device
Implicit <i>(deprecated)</i>	Formerly used for SPAs; superseded by Authorization Code + PKCE

Choose the grant type based on **who** is authenticating: a human user (Authorization Code + PKCE), a background service (Client Credentials), or a constrained device (Device Code).

OpenID Connect (OIDC) On-Premise

What is OIDC?

- Authentication layer built on top of OAuth 2.0.
- Issues ID tokens with user identity claims.

Common uses:

- Single Sign-On across multiple internal applications.
- Centralized identity management within organizations.

OIDC Authentication Flow (Simplified)

1. User initiates authentication from app.
2. Redirected to OIDC provider.
3. User authenticates (LDAP/SAML backend).
4. Provider issues ID token (JWT) containing identity claims.
5. Application validates token and extracts user information.

Example OIDC ID Token (JWT)

```
{  
  "iss": "https://oidc.example.com",  
  "sub": "user123",  
  "aud": "internal-app",  
  "exp": 1712312345,  
  "iat": 1712308745,  
  "name": "John Doe",  
  "email": "john.doe@example.com"  
}
```

OIDC Implementation Best Practices

- ✓ Use secure algorithms (RS256 recommended).
- ✓ Short-lived ID tokens with refresh tokens.
- ✓ Validate token claims strictly (issuer, audience, expiry).
- ✓ Secure handling and storage of tokens.

Common On-Prem Implementation Pitfalls

- Misconfigured SSL/TLS or certificates.
- Poor token lifecycle management.
- Improper permissions in directory services.
- Inadequate logging and monitoring.

Practical Checklist for Implementation

- ✓ Centralize identity management (LDAP, IdP).
- ✓ Strictly adhere to security standards (SSL/TLS, PKCE).
- ✓ Regularly audit configurations and permissions.
- ✓ Ensure comprehensive logging and monitoring.

Summary of Practical Implementations

- LDAP centralizes user management securely.
- SAML enables secure internal SSO.
- OAuth/OIDC offer robust, modern authentication options.
- Effective integration requires strict adherence to best practices.

Common Implementation Issues and Security Best Practices

Common Implementation Issues

- Misconfigured authentication servers.
- Poorly managed cryptographic keys and certificates.
- Insecure session and token management.
- Insufficient logging and monitoring.
- Lack of proper input validation.

Misconfigured Authentication Servers

Common mistakes:

- Unsecured connections (missing SSL/TLS).
- Weak default settings or credentials.
- Incorrect permissions on sensitive directories.

Best practices:

- ✓ Regularly review configuration files.
- ✓ Always use secure protocols (HTTPS, LDAPS).
- ✓ Regular vulnerability assessments.

Cryptographic Keys and Certificate Management

Common pitfalls:

- Weak or reused keys.
- Expired or self-signed certificates.
- Improper storage of keys.

Best practices:

- ✓ Regularly rotate keys and certificates.
- ✓ Store keys securely (Hardware Security Modules - HSMs).
- ✓ Monitor certificate expiration proactively.

Insecure Session and Token Handling

Common pitfalls:

- Unsecured cookies (missing `Secure`, `HttpOnly` flags).
- Excessively long-lived tokens.
- Storing sensitive data in JWTs.

Best practices:

- ✓ Secure all cookies (use flags: `Secure`, `HttpOnly`).
- ✓ Implement short-lived tokens and use refresh tokens securely.
- ✓ Limit token payloads to necessary, non-sensitive data.

Insufficient Logging and Monitoring

Common mistakes:

- Minimal or no logging of authentication events.
- Failure to detect unauthorized access attempts.
- Logs stored insecurely or unencrypted.

Best practices:

- ✓ Comprehensive logging of all authentication events.
- ✓ Monitor for anomalous activities and alert promptly.
- ✓ Secure and regularly audit log storage.

Lack of Input Validation

Common vulnerabilities:

- Injection attacks (LDAP, XML, SQL injection).
- Maliciously crafted tokens and payloads.

Best practices:

- ✓ Strictly validate and sanitize all inputs.
- ✓ Use whitelisting rather than blacklisting.
- ✓ Adopt frameworks with built-in protections.

Security Implementation Checklist

- ✓ **Secure Communication:** Always use encrypted channels (SSL/TLS).
- ✓ **Key Management:** Securely manage and rotate keys regularly.
- ✓ **Session & Tokens:** Implement secure session handling (flags, rotation).
- ✓ **Logging:** Maintain detailed audit logs and real-time monitoring.
- ✓ **Input Validation:** Enforce rigorous input validation throughout systems.

Summary of Best Practices

- Regular audits and configuration reviews.
- Comprehensive and secure logging.
- Proper session and token lifecycle management.
- Strong cryptographic practices and key management.
- Ongoing security training and awareness.

Real-world Case Studies

Case Study 1: Equifax Data Breach (2017)

Incident Overview:

- Massive breach due to a vulnerable web portal.
- Compromised personal information of ~147 million people.

Root Causes:

- Failure to patch a known Apache Struts vulnerability (CVE-2017-5638).
- Insufficient network segmentation and monitoring.
- Poor access control and excessive data retention.

Lessons from Equifax

- ✓ **Patch Management:** Timely updates and patching of vulnerabilities.
- ✓ **Strong Session Handling:** Robust token/session controls.
- ✓ **Monitoring:** Early detection through effective monitoring.

Case Study 2: Colonial Pipeline Ransomware Attack (2021)

Incident Overview:

- Ransomware attack halted fuel distribution across the US East Coast.
- Attack initiated via a legacy VPN account using single-factor authentication.

Root Causes:

- Single-factor (password-only) authentication on a legacy VPN account.
- Weak password management and no MFA enforcement on critical access.

Lessons from Colonial Pipeline

- ✓ **Multi-Factor Authentication:** Implement MFA for critical access.
- ✓ **Credential Security:** Enforce strong password policies and monitoring.
- ✓ **Zero Trust Principles:** Implement least-privilege access controls.

Case Study 3: SolarWinds Supply Chain Attack (2020)

Incident Overview:

- Attackers compromised authentication processes via software updates.
- Unauthorized access to sensitive networks, including government and corporate systems.

Root Causes:

- Weak code signing practices.
- Poor internal authentication controls.
- Inadequate monitoring of authentication activities.

Lessons from SolarWinds

- ✓ **Secure Software Supply Chain:** Rigorous control of software updates.
- ✓ **Strong Internal Authentication:** Multi-layered authentication and access controls.
- ✓ **Continuous Monitoring:** Early detection through advanced monitoring systems.

Common Patterns from Case Studies

- Weak or single-factor authentication.
- Poorly managed sessions or tokens.
- Lack of robust monitoring and logging.
- Failure to adhere to Zero Trust principles.

Practical Takeaways

- ✓ Implement multi-factor authentication across critical services.
- ✓ Regularly audit and update access controls.
- ✓ Prioritize patching and vulnerability management.
- ✓ Strengthen monitoring, detection, and response capabilities.

Summary

- Real-world breaches underline importance of robust authentication and authorization.
- Learn from mistakes and proactively secure systems.
- Continuously improve practices based on evolving threats.

Open Source Self-Hosted Identity Providers (IdP)

Why Choose Self-Hosted IdPs?

- **Complete control** of identity management.
- Better **data privacy** and compliance.
- **Customization** and integration flexibility.

Popular Open Source IdPs

- **Authentik** (*highly recommended*)
- Keycloak
- IdentityServer
- Gluu Server

Comparison of IdPs

Feature	Authentik	Keycloak	IdentityServer	Gluu Server
LDAP Integration	✓	✓	✗ (limited)	✓
SAML Integration	✓	✓	✗	✓
OAuth/OIDC	✓	✓	✓	✓
MFA Support	✓	✓	✗ (limited)	✓
Deployment Ease	High	Medium	Medium-Low	Low
Community & Docs	Active	Very Active	Medium	Medium

Spotlight: **Authentik**

- Highly flexible and easy-to-manage identity solution.
- Supports multiple protocols:
 - LDAP
 - OAuth 2.0
 - OpenID Connect (OIDC)
 - SAML
 - MFA (TOTP, WebAuthn)
- Modern, user-friendly interface.

Security Best Practices with Authentik

- ✓ Secure connections using HTTPS/SSL certificates.
- ✓ Rotate keys and secrets regularly.
- ✓ Limit access to administrative UI to authorized personnel only.
- ✓ Backup regularly to protect user data and configurations.

Why Authentik?

- Modern, intuitive, flexible.
- Ideal for quick deployment in on-prem infrastructures.
- Comprehensive documentation and active development.
- Strong built-in support for key authentication standards.

Deploying Authentik (Docker Example)

Docker Compose Example (*simplified – replace all placeholder values before use*):

```
# ⚠️ Replace AUTHENTIK_SECRET_KEY and all passwords before ANY deployment!  
# Pin the image tag to a specific version (e.g., 2024.2.2) instead of 'latest' in production.
```

```
services:  
  server:  
    image: ghcr.io/goauthentik/server:2024.2.2  
    command: server  
    ports:  
      - "9000:9000"  
      - "9443:9443"  
    environment:  
      AUTHENTIK_SECRET_KEY: "CHANGE_ME_run_openssl_rand_-base64_60"  
      AUTHENTIK_REDIS_HOST: redis  
      AUTHENTIK_POSTGRES_HOST: db  
      AUTHENTIK_POSTGRES_USER: authentik  
      AUTHENTIK_POSTGRES_PASSWORD: "CHANGE_ME"  
      AUTHENTIK_POSTGRES_NAME: authentik  
    depends_on: [db, redis]
```

```
worker:
  image: ghcr.io/goauthentik/server:2024.2.2
  command: worker
  environment:
    AUTHENTIK_SECRET_KEY: "CHANGE_ME_run_openssl_rand_-
base64_60"
    AUTHENTIK_REDIS_HOST: redis
    AUTHENTIK_POSTGRESQL_HOST: db
    AUTHENTIK_POSTGRESQL_USER: authentik
    AUTHENTIK_POSTGRESQL_PASSWORD: "CHANGE_ME"
    AUTHENTIK_POSTGRESQL_NAME: authentik
  depends_on: [db, redis]

redis:
  image: redis:alpine

db:
  image: postgres:16-alpine
  environment:
    POSTGRES_USER: authentik
    POSTGRES_PASSWORD: "CHANGE_ME"
    POSTGRES_DB: authentik
  volumes:
    - pg_data:/var/lib/postgresql/data

volumes:
  pg_data:
```

Initial Setup Steps

1. Launch Authentik:

```
docker compose up -d
```

2. Access UI at:

```
https://localhost:9443
```

3. Log in initially with:

- Username: `akadmin`
- Password: Generated and displayed at first launch logs.

Practical Example: Configuring LDAP in Authentik

Step-by-Step:

1. Navigate to: **Applications** → **Create** → **LDAP Provider**

- Name: `Internal LDAP`
- Base DN: `dc=example,dc=com`

2. **Configure Users and Groups:**

- User DN: `ou=users,dc=example,dc=com`
- Group DN: `ou=groups,dc=example,dc=com`

3. **Bind User Configuration:**

- Set service account credentials for binding.

4. **Enable Provider** and save configuration.

Practical Example: SAML Application Integration

Step-by-Step:

1. Create SAML Provider:

- Go to: **Applications → Create → SAML Provider**
- Set ACS URL provided by Service Provider (SP).
- Define Entity ID (e.g., `https://sp.example.com/metadata`).

2. SAML Assertions & Attributes:

- Set attribute mappings (e.g., username, email, roles).

3. Service Provider (SP) Configuration:

- Provide SP with Authentik's Metadata URL:

```
https://your-authentik-domain/metadata
```

Practical Example: OAuth/OIDC Integration

Step-by-Step:

1. Create OAuth Provider:

- Navigate: **Applications** → **Create** → **OAuth2 Provider**
- Set allowed redirect URIs (e.g.,
`https://app.example.com/callback`).

2. Define Scopes & Claims:

- Choose standard scopes (`openid`, `email`, `profile`).
- Map required claims (user identity details).

3. Application Configuration:

- Use provided Client ID and Secret in your app.
- Configure app to authenticate via Authentik OIDC endpoint:

```
https://your-authentik-domain/application/o/authorize/
```

Configuring MFA in Authentik

Enable TOTP/WebAuthn MFA:

- Navigate: **Flows** → **Default Authentication Flow**
- Insert MFA Stage (TOTP/WebAuthn) into flow.
- Users enroll MFA on next login.

Practical Example:

- User logs in → Prompted to enroll MFA (QR Code or WebAuthn).
- Subsequent logins require MFA verification.

Summary & Practical Takeaways

- **Authentik** provides a powerful, easy-to-use IdP solution.
- Simple configuration across LDAP, SAML, OAuth/OIDC.
- MFA and advanced security features built-in.
- Recommended for secure, robust internal authentication and authorization.

Next Steps

- Deploy and evaluate Authentik in a test environment.
- Explore official documentation: **goauthentik.io**
- Implement best practices discussed today.
- **YT: Authentik Setup by Cooptonian**

Summary and Q&A

Recap of Key Points

- **Authentication** verifies **identity** ("Who are you?").
- **Authorization** controls **access and permissions** ("What can you access?").

Authentication Methods

- **Passwords:** Use strong hashing (bcrypt, Argon2) and salting; check against breached-password lists.
- **Multi-Factor Authentication (MFA):** Significantly enhances security over passwords alone.
- **Passkeys (FIDO2/WebAuthn):** Phishing-resistant, hardware-backed, passwordless credentials.
- **Single Sign-On (SSO):** Simplifies user experience; reduces password fatigue.

Authorization Models

Model	Use Case
DAC	User-controlled permissions
MAC	High-security environments
RBAC	Enterprise and role-based access
ABAC	Dynamic and context-aware access

Practical Implementation

- **LDAP + Kerberos:** Centralized directory management and network authentication.
- **SAML:** XML-based SSO for internal enterprise services.
- **OAuth 2.0/OIDC:** Delegated authorization and modern identity management.

Session & Token Management

- Secure sessions (flags: `HttpOnly`, `Secure`).
- JSON Web Tokens (JWT): Stateless and scalable.
- Always encrypt communications (HTTPS).

Best Practices and Pitfalls

- Regularly audit configurations and certificates.
- Secure cryptographic keys and tokens.
- Implement rigorous logging, monitoring, and input validation.

Advanced Topics

- **Biometrics:** Convenient and phishing-resistant, but biometric data cannot be "changed" if compromised — requires careful storage and liveness detection.
- **Zero Trust Architecture:** "Never trust, always verify" — authenticate and authorize every request, even from inside the network. Defined by NIST SP 800-207.
- **Continuous Authentication:** Verifies identity throughout a session (e.g., behavioral biometrics, typing patterns) rather than only at login.
- **AI-driven Authentication:** Uses machine learning to detect anomalous behavior and adjust trust dynamically (risk-based / adaptive authentication).
- **Passkeys / FIDO2:** Hardware-backed, phishing-resistant credentials that replace passwords entirely — natively supported by Apple, Google, and Microsoft.

Real-world Lessons

- Weak authentication and poor practices lead to breaches.
- Strong controls, timely patching, and robust monitoring prevent incidents.

Interactive Q&A

- **Your Questions:** Clarifications, practical concerns, implementation scenarios.
- **Discussion:** Share experiences, challenges, and insights.

Final Thoughts

- Security evolves constantly—stay informed.
- Implement layered security ("Defense-in-Depth").
- Regularly review, adapt, and improve authentication and authorization practices.

Thank You!

Self test

1. What is the primary difference between authentication and authorization?
2. Describe three common methods of authentication and their strengths and weaknesses.
3. What is Multi-Factor Authentication (MFA), and why is it considered more secure than password-only authentication?
4. Explain Single Sign-On (SSO). What is a potential security risk associated with SSO?
5. Name and briefly describe four common authorization models.
6. Compare and contrast Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC). Which scenarios are each best suited for?
7. What are JSON Web Tokens (JWT), and how are they typically used in token-based authentication?
8. List three common security pitfalls in session management and describe best practices to mitigate each.
9. In the context of on-premise infrastructure, describe how LDAP and SAML can work together to manage identity and access control.

10. Using the Equifax data breach as an example, discuss key security lessons related to authentication and authorization practices.
11. What are passkeys (FIDO2/WebAuthn)? How do they differ from password-based and OTP-based MFA, and why are they considered phishing-resistant?

Speaker notes